

72.39 - Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial - Stage I



Integrantes:

- Matías Reynoso Fandiño (61402)
- Santiago Patiño (65549)
- Juan Ferrin (65135)
- Felipe Merlo (65736)

Fecha de Entrega: 06/09/2026

1. Equipo

Nombre	Apellido	Legajo	E-mail
Matías	Reynoso Fandiño	61402	mreynoso@itba.edu.ar
Santiago	Patiño	65549	spatino@itba.edu.ar
Juan	Ferrin	65135	jferrin@itba.edu.ar
Felipe	Merlo	65736	fmerlo@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/matreynoso/boardLang>.

3. Dominio

El dominio elegido es el de los juegos de tablero de losetas cuadradas por turnos, de información completa y dos o más jugadores, tomando como referentes centrales el ajedrez, las damas y variantes emparentadas (por ejemplo, juegos de “captura la bandera” sobre tablero). Estos juegos comparten una misma estructura conceptual: un tablero de casillas discretas dispuestas en filas y columnas; un conjunto de jugadores que se alternan turnos según un orden fijo; piezas que ocupan casillas y que se desplazan o capturan de acuerdo a reglas propias de cada tipo; y una condición de victoria que determina cuándo y cómo termina la partida.

El lenguaje propuesto es declarativo y se estructura mediante tres construcciones de nivel superior (game, piece y player) evitando que el usuario deba programar imperativamente el tablero, el ciclo de turnos o la validación de movimientos. En lugar de eso, un programa describe cómo es el tablero, qué puede hacer cada pieza y cómo se gana, y es el compilador quien se encarga de darle semántica ejecutable a esa descripción.

El dominio fue diseñado para ser suficientemente general tal que se puedan construir variantes atípicas de varios juegos clásicos. Algunas características que reflejan esta idea son la definición del tablero (cuyo tamaño es configurable, puede o no tener condiciones de contorno, y puede admitir casillas bloqueadas), el número y alternancia de los jugadores (cantidad elegida arbitrariamente, secuencia de orden personalizable que permite asimetría en cantidad de turnos de cada jugador por ciclo), la definición de piezas (cuyas restricciones de movimientos y ataques son propias, independientes entre sí y muy genéricas) y la condición de victoria (que permite juegos orientados a la conquista de territorio, a la carrera, a la captura de piezas estratégicas o a la batalla masiva).

Quedan fuera del alcance del dominio y se consideran para extensiones futuras las reglas que requieren estado o memoria histórica de la partida (jaque y jaque mate, enroque, doble paso inicial del peón, captura obligatoria encadenada).

Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

(I). Construcciones de nivel superior

- (I.1). **Game** (exactamente una por programa): configuración global — dimensiones del tablero, condiciones de contorno, casilleros bloqueados, cantidad e identidad de jugadores y orden de turno, cantidad de movimientos por turno, y condición de victoria.
- (I.2). **Piece** (cero o más): define un tipo de pieza — cómo se mueve (moves), cómo captura (attacks, opcional), a qué se promociona y dónde (promote, opcional) y cuánto vale al ser capturada (value, opcional). Un tipo es compartido por todos los jugadores.
- (I.3). **Player** (una por participante): orientación de “adelante” (front) y la disposición inicial de sus piezas (placement), casillero de llegada para cumplir condición de victoria propia, si aplicase.

Las tres construcciones son independientes, pueden declararse en cualquier orden dentro del programa y cada una es un bloque delimitado por llaves.

(II). Expresividad del movimiento

- (II.1). Composición de pasos con “&”: encadena pasos en un único movimiento articulado que no verifica casilleros intermedios (ej. el “salto en L” del caballo).
- (II.2). Alternativas de movimiento con “;”: una pieza puede tener varios patrones de movimiento independientes entre los que el jugador elige uno.
- (II.3). Conjuntos de dirección con “|”: agrupa varias orientaciones (front/back/left/right) dentro de un mismo paso.
- (II.4). Precedencia bien definida entre operadores: | (más fuerte) → & → , (más débil), lo que permite escribir expresiones compuestas sin paréntesis.
- (II.5). Dos modos de desplazamiento: slide (se detiene al chocar con una pieza o un bloqueo) y jump (ignora piezas intermedias); toda cadena con “&” es jump de forma implícita.
- (II.6). Marcos de referencia relativos: en un movimiento simple, front/back/left/right son relativos al jugador (y definidos en la construcción *player*); a partir del segundo paso de una cadena &, son relativos al rumbo del paso anterior.

(III). Condiciones y modelos de captura

- (III.1). Predicados de casillero target (casilla de destino) y over (casillas intermedias), combinables con and, y con ocupación empty / enemy / ally.
- (III.2). Inferencia automática del modelo de captura según el predicado usado: target enemy implica captura por reemplazo (estilo ajedrez); over enemy implica captura por salto (estilo damas), exigiendo destino vacío.
- (III.3). Valores por defecto que reducen la verbosidad: moves sin when equivale a when target empty; attacks sin when equivale a when target enemy; una pieza sin bloque attacks captura reutilizando su propia expresión de moves.

- (IV). Reglas de turno y de partida
 - (IV.1). Orden de turno cíclico definido por la lista `players`, donde un nombre repetido otorga turnos adicionales a ese jugador (permite generalizar a N jugadores con proporciones distintas).
 - (IV.2). `moves-per-turn`: cantidad de movimientos legales e independientes que el jugador activo realiza en su turno (default 1), ortogonal al orden de turno.
 - (IV.3). Promoción de piezas (`promote`) al alcanzar una región del tablero (`last-row`, `first-row` o una posición puntual).
 - (IV.4). Sistema de puntaje opcional (value en cada pieza, default 1) utilizado por la condición de victoria `points`.
 - (IV.5). Cinco condiciones de victoria: `capture N <pieza>`, `capture-all`, `points N`, `reach <posición> [with <pieza>]` (meta compartida) y `reach goal [with <pieza>]` (meta propia por jugador, declarada en su bloque `player`), más `custom` como punto de extensión; por defecto, un jugador sin movimientos legales en su turno pierde la partida.
 - (IV.6). Tableros irregulares mediante `blocked`: los casilleros bloqueados no son alcanzables ni ubicables al inicio, y actúan como pared para los movimientos `slide`.
- (V). Prestaciones léxicas y de escritura
 - (V.1). Comentarios de línea (`// ...`) y de bloque (`/* ... */`, no anidables), para documentar programas.
 - (V.2). Resolución explícita de la colisión entre identificadores y posiciones: todo token con forma `[a-z][0-9]+` se reconoce como posición, restringiendo qué nombres puede tener una pieza o un jugador.
 - (V.3). Un conjunto amplio de valores por defecto que permite escribir programas concisos, declarando explícitamente solo aquello que se aparta del comportamiento estándar.

4. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Bloque `game` mínimo

```
game {  
  board: 8 x 8  
  players: rojo, azul  
  win: capture-all  
}
```

Verifica el parseo de un bloque `game` con sus campos básicos: tablero, roster de jugadores y condición de victoria.

(II). Pieza con deslizamiento infinito

```
piece torre {  
  moves: slide inf straight  
}
```

Verifica distance = inf y kind = slide explícito, junto a una base straight sin orientación (las 4 direcciones ortogonales).

(III). Movimiento y ataque declarados por separado

```
piece peon {  
  moves: 1 front straight  
  attacks: 1 front diagonal  
}
```

Verifica que moves y attacks puedan coexistir con sus condiciones implícitas por defecto (target empty y target enemy, respectivamente).

(IV). Movimiento articulado con &

```
piece caballo {  
  moves: jump 2 straight & 1 (left|right) straight  
}
```

Verifica la composición de pasos con &, el conjunto de direcciones con | y el cambio de marco de referencia en el segundo paso de la cadena.

(V). Alternativas de movimiento con coma

```
piece dama {  
  moves: slide inf straight, slide inf diagonal  
}
```

Verifica el operador , como alternativa entre dos move_term completos e independientes.

(VI). Condición compuesta target/over

```
piece hombre {  
  attacks: jump 2 front diagonal when target empty and over enemy  
}
```

Verifica square_pred compuestos unidos con and, utilizados para inferir el modelo de captura por salto.

(VII). Promoción de pieza

```
piece peon {  
  promote: dama at last-row  
}
```

Verifica la cláusula promote con una región simbólica (last-row).

(VIII). Bloque player con orientación, meta y ubicaciones

```
player rojo {  
  front: up  
  goal: h8  
  corredor at a1, b1, c1  
}
```

Verifica que player admita front, goal y una o más cláusulas de placement en cualquier combinación.

(IX). Tablero irregular y reglas de turno no triviales

```
game {  
  board: 8 x 8  
  blocked: d4, e4, d5, e5  
  moves-per-turn: 2  
  on-capture: continue  
  win: points 3  
  edges: wrap  
}
```

Verifica blocked, moves-per-turn, on-capture y win: points combinados en un mismo bloque game. *Edges: wrap* elimina la condición de contorno (quita los bordes).

(X). Victoria por posición compartida y comentarios

```
/* partida corta */  
game {  
  board: 8 x 8  
  players: rojo, azul // el orden es el de turno  
  win: reach e4 with corredor  
}
```

Verifica win: reach con la cláusula with, y que los comentarios de línea y de bloque sean correctamente descartados por el analizador léxico.

Además, los siguientes casos de prueba de **rechazo**:

(XI). Identificador con forma de posición

```
Categoría: léxico
piece a2 {
  moves: 1 front straight
}
```

Se rechaza porque a2 coincide con la forma de posición [a-z][0-9]+, que tiene prioridad léxica sobre identificar; ningún identificador puede tener esa forma.

(XII). Falta el separador ':' en un campo

```
Categoría: sintáctico
piece torre {
  moves slide inf straight
}
```

Se rechaza porque piece_clause exige el token ':' entre la palabra clave moves y el move_expr que la sigue.

(XIII). Dirección sin base

```
Categoría: sintáctico
piece p {
  moves: 1 front
}
```

Se rechaza porque direction exige un base obligatorio (straight | diagonal); front es solo el orient_set opcional y no alcanza por sí solo.

(XIV). Pieza referenciada pero nunca declarada

```
Categoría: semántico
player rojo {
  torre at a1, h1
}
```

Se rechaza porque la torre no fue declarada con un bloque piece en el programa. En la entrega de Stage II podría producir un falso positivo, tal como anticipa el enunciado, hasta contar con chequeo semántico completo.

(XV). Bloque game sin condición de victoria

```
Categoría: semántico
game {
  board: 8 x 8
  players: rojo, azul
}
```

Se rechaza porque la especificación exige exactamente un win por partida; la gramática libre de contexto lo admite ausente, pero el análisis semántico lo declara obligatorio (lo mismo aplica, en sentido inverso, a declarar más de un bloque game).

5. Ejemplos

En el **Cód. (6.1)**, se puede ver como se define un único movimiento compuesto por dos pasos unidos con &, lo que lo convierte automáticamente en jump (ignora las piezas intermedias). El primer paso, sin orientación, cubre las 4 direcciones ortogonales tomando como referencia al jugador; el segundo paso usa el conjunto (left|right), interpretado relativo al rumbo del paso anterior, generando las 8 combinaciones en “L” propias del caballo de ajedrez.

```
piece caballo {
  moves: jump 2 straight & 1 (left|right) straight
}
```

Código 6.1: Pieza con movimiento compuesto.

En el **Cód. (6.2)**, se ilustra un juego de dos jugadores en el que rojo mueve dos veces por cada vez que mueve azul (su nombre aparece repetido en players), y una condición de victoria reach goal con meta propia por jugador: rojo gana llevando un corredor a f6 y azul llevando el suyo a a1, cada meta declarada en el bloque player correspondiente.

```
game {
  board: 6 x 6
  players: rojo, rojo, azul
  win: reach goal with corredor
}

player rojo {
  goal: f6
  corredor at a1
}

player azul {
  front: down
  goal: a1
  corredor at f1
}
```

Código 6.2: Ejemplo de juego con turnos irregulares y meta propia por jugador.